

5347 Count the Number of Complete Components | Math | Adjacency Matrix | DFS | DSU

Hash Table

Math

Depth-First Search

Union-Find

5+

```
class Solution {  
    static int V, E;  
  
    public int countCompleteComponents(int n, int[][] edges) {  
        List<Integer>[] A = new ArrayList[n];  
        Arrays.setAll(A, _ -> new ArrayList<>());  
  
        for (int[] e : edges) {  
            A[e[0]].add(e[1]);  
            A[e[1]].add(e[0]);  
        }  
  
        boolean[] vis = new boolean[n];  
        int res = 0;  
  
        for (int i = 0; i < n; i++) {  
            boolean state = vis[i];  
  
            if (!state) {  
                V = 0; E = 0;
```

```

        dfs(i, A, vis);

        if (E == V * (V - 1)) res++;
    }
}

return res;
}

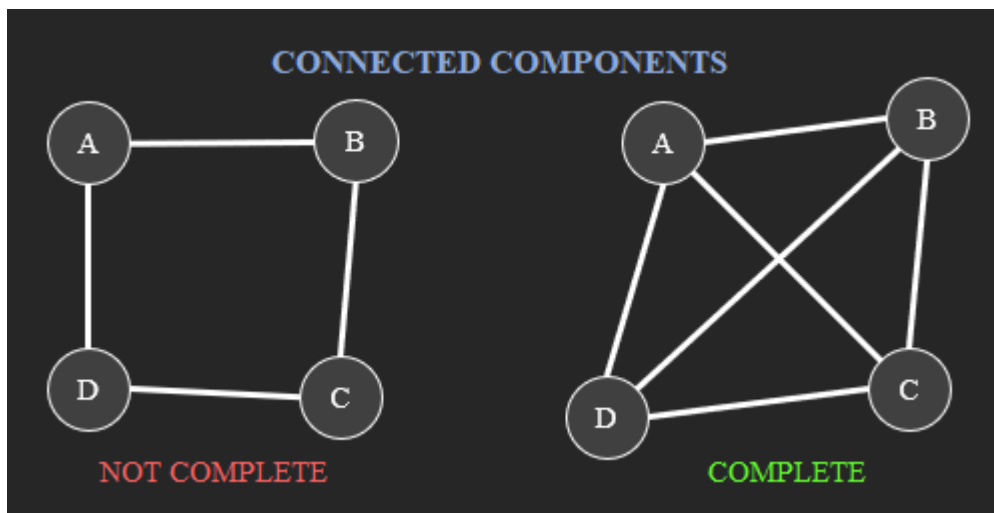
private void dfs(int x, List<Integer>[] A, boolean[] vis) {
    V++;
    E += A[x].size();
    vis[x] = true;

    for (int state : A[x])
        if (!vis[state])
            dfs(state, A, vis);
}
}

```

Depth-First Search

A connected component is if every vertex is connected to every other vertex.



traverses all vertices belonging to the same connected component, making it easy to inspect one component at a time.

Complete Component Check

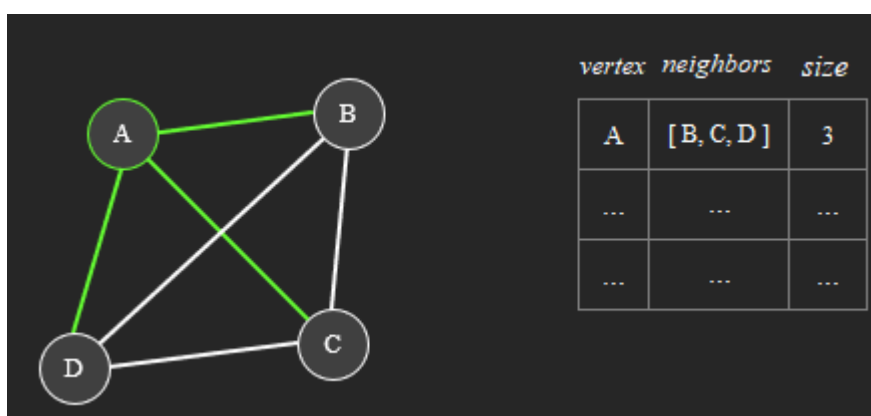
A complete graph contains an edge between every pair of vertices.

The number of distinct pairs that can be formed from n vertices is:

which is the number of edges in a complete graph.

During the DFS, we collect the degree (the number of neighbors) of every vertex.

The adjacency array stores all neighbors of vertex v , so its size is simply the degree of the current vertex.

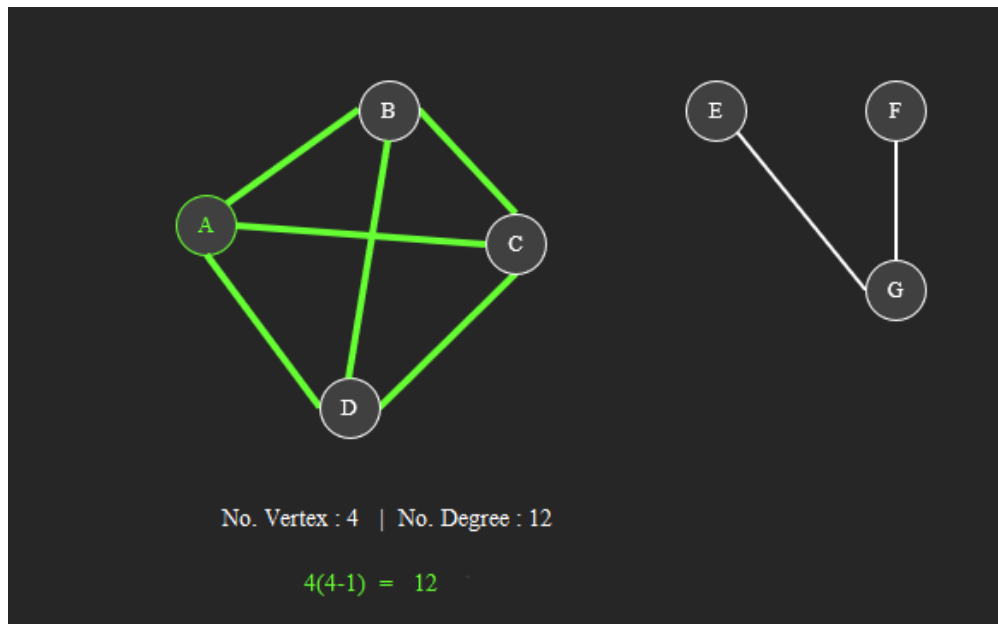


Since every edge is counted as a degree for both connected vertices, the degree count is:

Thus, the only thing we need to accumulate:

- The count of vertices, .
- The sum of the degrees of all vertices, .

After the search, the containing the current has been fully explored.



We can determine if this is a if:

If this is , then we increase the .

Time Complexity: $O(n + m)$

Space Complexity: $O(n + m)$

Disjoint Set Union (DSU)

```
class Solution {
```

```
public:
```

```
int countCompleteComponents(int n, vector<vector<int>>& edges) {
```

```
    vector<int> root(n);
```

```
    iota(root.begin(), root.end(), 0);
```

```
    auto find = [&](this auto& self, int i) -> int {
```

```

        return root[i] == i ? i : root[i] = self(root[i]);
    };

    for (auto& e : edges)
        root[find(e[0])] = find(e[1]);

    vector<int> V(n, 0);
    vector<int> E(n, 0);
    for (int i = 0; i < n; i++)
        V[find(i)]++;
    for (auto& e : edges)
        E[find(e[0])]++;
    int res = 0;
    for (int i = 0; i < n; i++)
        res += (V[i] && E[i] == V[i] * (V[i] - 1) >> 1);

    return res;
}
};

```

Instead of traversing the graph, we can use to group vertices that belong to the same connected component.

- After all connected , we count the number of vertices and edges belonging to each component.

Similar reasoning to: [Minimum Score of a Path Between Two Cities](#)

- A complete component with n must contain exactly:

So we compare the collected edge count against this value.

- If they , the component is complete.

Time Complexity: $O(n + m)$

Space Complexity: $O(n)$